

RAG 테스트

1. RAG

(Retrieval Augmented Generation)

RAG(Retrieval-Augmented Generation, 검색기반 답변 생성)

- RAG는 사용자의 질문에 바로 답하지 않고, 먼저 관련 문서나 데이터에서 필요한 정보를 찾은 뒤 그 내용을 바탕으로 답변을 만드는 방식입니다.
- 일반 LLM은 주로 학습 당시의 지식에 의존하지만, RAG를 사용하면 최신 문서, 사내 매뉴얼, 정책 자료, 기술 문서, 업무 데이터 등을 함께 참고할 수 있습니다.
- 따라서 RAG는 답변의 근거를 강화하고, 부정확한 답변이나 할루시네이션을 줄이는 데 도움이 됩니다.

RAG의 기본 흐름

정보 검색

- 사용자의 질문과 관련된 자료를 벡터 DB, 검색 엔진, 문서 저장소, 지식베이스 등에서 찾습니다.

참고 문맥

- 구성검색된 자료 중 답변에 필요한 내용을 선별하여 프롬프트에 함께 포함합니다.

답변 생성

- LLM은 검색된 자료를 참고하여 사용자의 질문에 맞는 답변을 생성합니다.

RAG가 필요한 경우

- 모델이 학습하지 않은 최신 정보가 필요한 경우
- 사내 문서, 업무 매뉴얼, 정책, 기술 자료를 근거로 답변해야 하는 경우
- 답변의 출처나 근거를 함께 제시해야 하는 경우
- 잘못된 답변을 줄이고 정확도를 높이고 싶은 경우

Retrieval Augmented Generation, 검색 증강 생성

- 일반 프롬프트

- 일리야 수츠케버는 어디에서 일하나요?

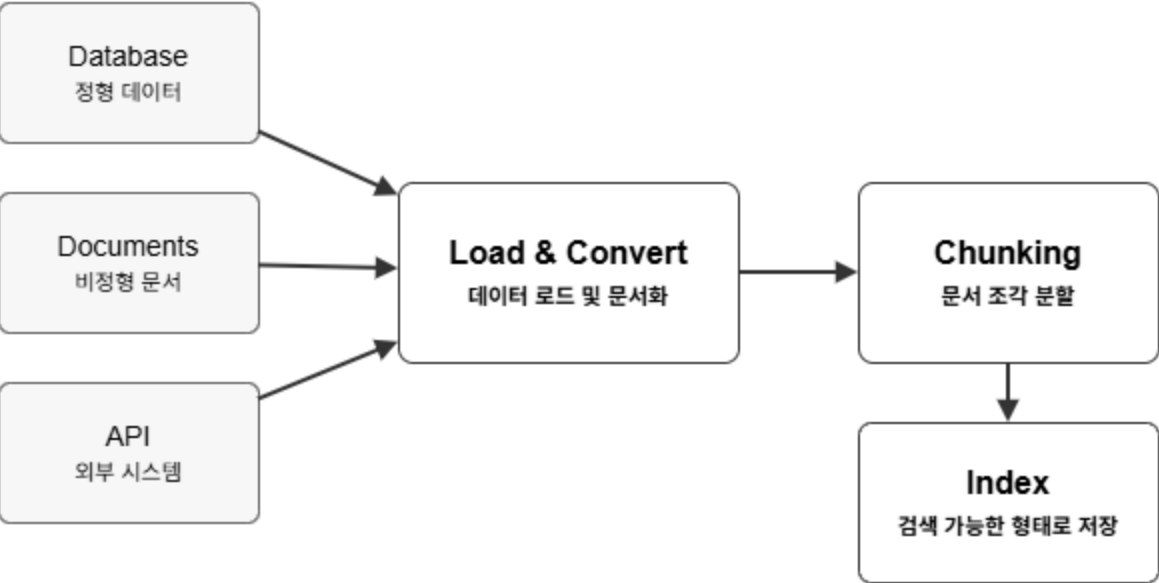
- RAG가 적용된 프롬프트

- 이 대화에 참고할 수 있는 관련 문맥은 다음과 같습니다.
 - """"{위키피디아 문서 내용 붙여넣기: https://en.wikipedia.org/wiki/Ilya_Sutskever}""""
 - 아래 질문에 대해, 위 문서를 근거로 답변해 주세요. 일리야 수츠케버는 어디에서 일하나요?

RAG의 두 단계

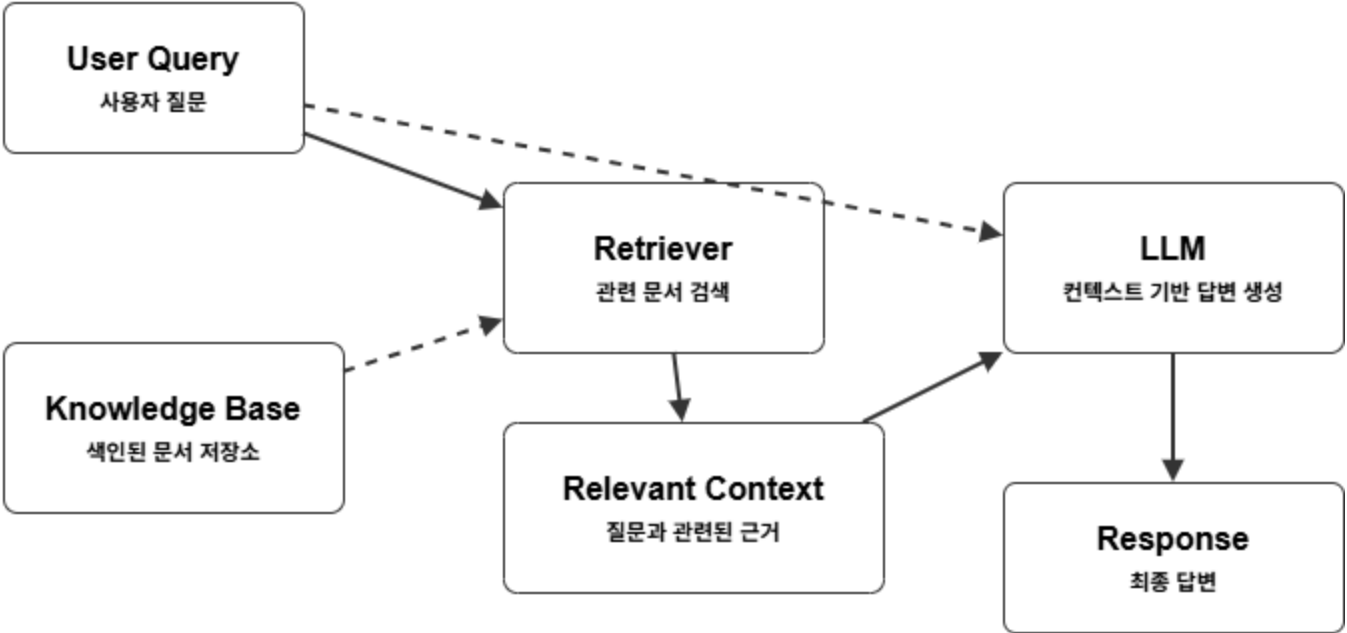
RAG는 문서를 미리 색인해 두는 단계와, 질문 시 관련 문서를 찾아 LLM 답변에 활용하는 단계로 나눌 수 있습니다.

1단계. Indexing Knowledge Base를 준비하는 단계



- 다양한 데이터 소스에서 데이터를 가져옵니다.
- 가져온 데이터를 LLM이 참고할 수 있는 문서 형태로 변환합니다.
- 긴 문서는 작은 조각으로 나누고, 검색 가능한 색인(Index)을 만듭니다.
- 이 단계는 사용자가 질문하기 전에 미리 준비해 두는 과정입니다.

2단계. Query 검색된 컨텍스트를 활용해 LLM에게 질문하는 단계



- 강의 교안용 설명**
- 사용자의 질문을 기준으로 Knowledge Base에서 관련 문서를 검색합니다.
 - 검색된 문서 조각을 질문과 함께 프롬프트에 넣습니다.
 - LLM은 검색된 근거를 참고하여 답변을 생성합니다.
 - 이 단계는 사용자가 질문할 때마다 실행됩니다.

핵심 정리: Indexing은 자료를 미리 찾아볼 수 있게 정리하는 과정이고, Query는 질문이 들어왔을 때 관련 자료를 찾아 LLM 답변에 붙이는 과정입니다.

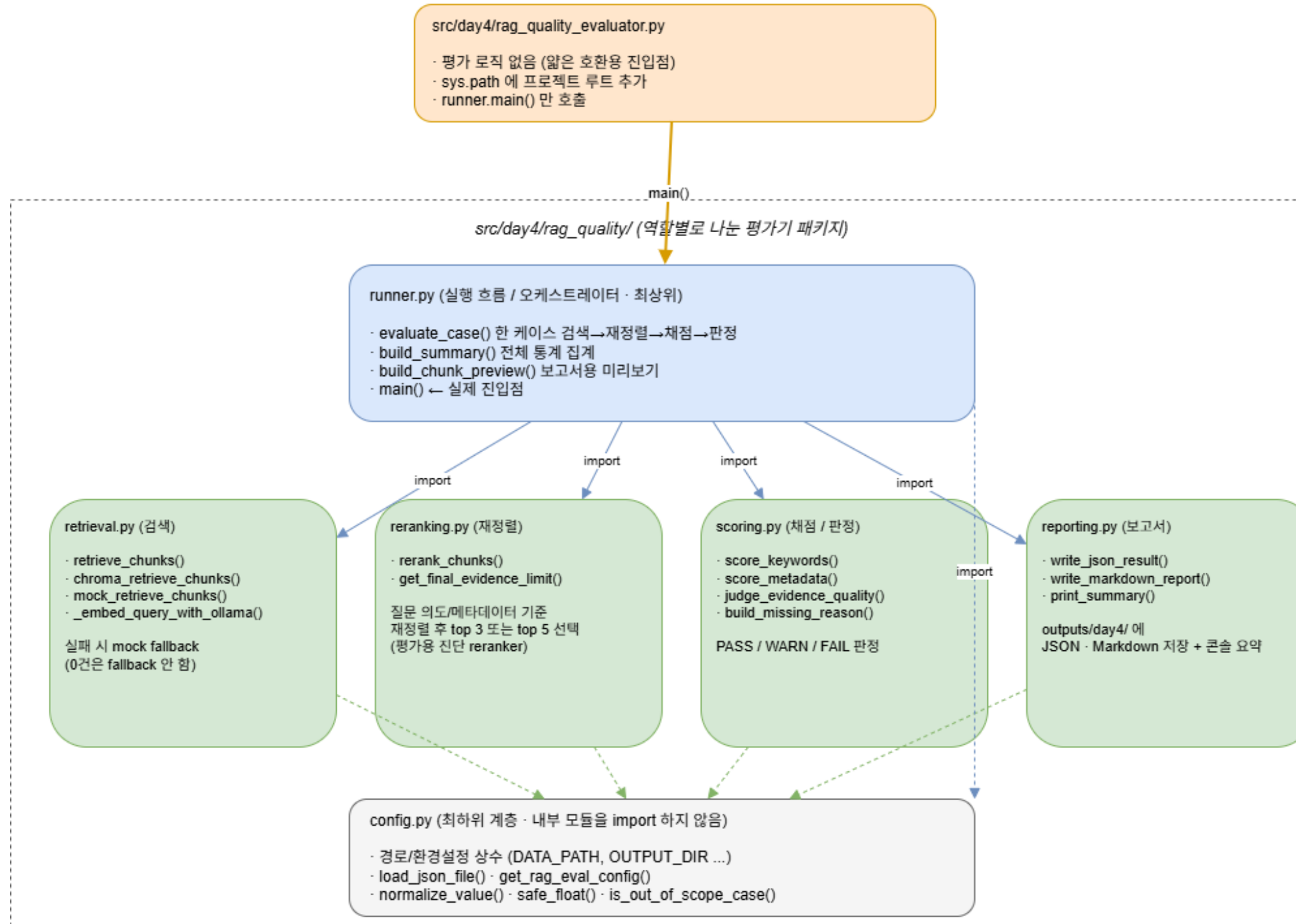
RAG 사용 시 고려할 점

- RAG는 검색된 자료를 근거로 답변을 생성하기 때문에 답변의 신뢰도를 높이는 데 도움이 됩니다.
- 하지만 RAG를 적용한다고 해서 모든 답변 문제가 자동으로 해결되는 것은 아닙니다.
- RAG는 정해진 문서나 데이터에 기반한 답변에는 강하지만, 새로운 아이디어를 자유롭게 제안하거나 창의적인 답변을 만드는 데에는 한계가 있을 수 있습니다.
- 검색된 문서를 프롬프트에 함께 넣어야 하므로 입력 토큰이 늘어나고, 그에 따라 처리 비용과 응답 시간이 증가할 수 있습니다.
- 문서 자체의 품질이 낮거나 문서 구조가 복잡하면, 모델이 내용을 잘못 해석하여 부정확한 답변을 생성할 수 있습니다.
- RAG의 성능을 높이려면 질문을 구체적으로 작성하고, 검색 결과를 명확하게 제공하며, 원하는 답변 형식과 기준을 함께 안내해야 합니다.

4. rag_quality_evaluator.py

RAG 평가 아키텍처

rag_quality 패키지 구조 & import 의존 계층



의존은 단방향: config(바닥) → 4개 중간 계층 → runner(최상위). 순환 없음.

RAG 검색 결과 해석 기준

- RAG 품질 평가는 단순히 “검색이 실행되었는가”를 확인하는 과정이 아닙니다. 검색된 문서가 질문과 충분히 관련 있는지, 필요한 근거가 누락되지 않았는지, 최종 답변에 적절히 반영되었는지를 함께 확인해야 합니다.

구분	평가 항목	확인 질문	확인 대상
검색 품질	관련성	검색된 문서가 질문의 알람 코드, 설비, 증상과 관련 있는가?	retrieved chunks, metadata
검색 품질	누락 여부	답변에 필요한 문서나 근거가 검색 결과에서 빠지지 않았는가?	retrieved chunks, result_count
검색 품질	순위 적절성	중요한 문서가 검색 결과 상위에 배치되었는가?	reranked results, scoring 결과
메타데이터 품질	metadata 일치	alarm_code, equipment_id, doc_name, section_title이 질문 내용과 일치하는가?	metadata, retrieved chunks
답변 품질	근거 반영	검색 결과가 최종 답변에 실제로 반영되었는가?	최종 답변의 근거 문장, report.md
안전한 응답	결과 부족 처리	검색 결과가 부족할 때 확정적으로 답하지 않고 한계를 설명했는가?	최종 답변, report.md

파일 구조

순서	파일	역할	호출 관계
1	rag_quality_evaluator.py	실행 진입점	직접 실행하면 runner.main()을 호출합니다. 실제 평가 로직은 거의 없습니다.
2	runner.py	전체 실행 흐름 제어	설정 읽기 → 케이스 읽기 → 검색 → 재정렬 → 채점 → 보고서 저장까지 오케스트레이션합니다.
3	config.py	공통 설정/유틸	입력 파일 경로, 출력 경로, Chroma 설정, top_k, mode 등을 제공합니다.
4	retrieval.py	검색 수행	Chroma 검색을 먼저 시도하고, 기술적 실패 시 mock 검색으로 fallback합니다.
5	reranking.py	검색 결과 재정렬	검색된 chunk를 질문 의도, metadata, 문서 유형 기준으로 다시 정렬합니다.
6	scoring.py	검색 품질 채점	keyword, metadata 매칭을 계산하고 PASS / WARN / FAIL을 판정합니다.
7	reporting.py	결과 저장/출력	JSON 결과, Markdown 보고서 저장, 콘솔 요약 출력을 담당합니다.

중요 함수

순서	핵심 함수	파일	하는 일
1	main()	runner.py	전체 평가 실행을 시작하고 흐름을 관리합니다.
2	get_rag_eval_config()	config.py	Chroma/mock 모드, DB 경로, top_k 설정을 읽습니다.
3	load_json_file()	config.py	평가할 질문 케이스 JSON을 읽습니다.
4	evaluate_case()	runner.py	케이스 1개에 대해 검색→재정렬→채점→판정을 수행합니다.
5	retrieve_chunks()	retrieval.py	질문에 맞는 문서 chunk를 검색합니다.
6	rerank_chunks()	reranking.py	검색된 chunk를 더 적절한 순서로 다시 정렬합니다.
7	judge_evidence_quality()	scoring.py	검색 결과가 좋은지 PASS / WARN / FAIL로 판정합니다.
8	write_markdown_report()	reporting.py	사람이 볼 수 있는 평가 보고서를 저장합니다.

5. Text to SQL

Text to SQL의 개념

- Text to SQL은 사용자가 자연어로 질문하면, AI가 그 질문을 SQL 쿼리로 변환하는 기술입니다.

- 예를 들어 사용자가 이렇게 질문하면,

최근 7일 동안 불량률이 가장 높은 설비를 알려줘

AI는 내부적으로 다음과 같은 SQL을 생성합니다.

```
SELECT equipment_id, defect_rate FROM quality_metrics
```

```
WHERE measured_at >= CURRENT_DATE - INTERVAL '7 days'
```

```
ORDER BY defect_rate DESC LIMIT 1;
```

핵심은 사람은 자연어로 질문하고, 시스템은 데이터베이스가 이해할 수 있는 SQL로 바꾼다는 점입니다.

Text to SQL이 필요한 이유

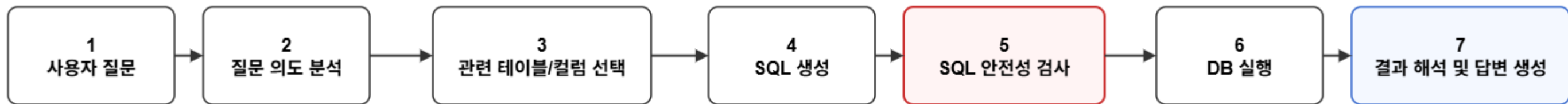
- 업무 현장에서는 데이터가 대부분 데이터베이스에 저장되어 있습니다.
- 하지만 모든 사용자가 SQL을 직접 작성할 수 있는 것은 아닙니다.
- Text to SQL을 사용하면 다음과 같은 장점이 있습니다.
 - 비개발자도 자연어로 데이터 조회 가능
 - 반복적인 SQL 작성 시간 감소
 - 데이터 분석 요청에 빠르게 대응 가능
 - Agent가 DB Tool과 연동해 업무 데이터를 직접 조회 가능
 - 제조 AI Agent에서는 설비 상태, 알람 이력, 품질 지표, 정비 이력 등을 자연어로 조회할 때 활용할 수 있습니다.

Text to SQL에서 중요한 설계 포인트

- Text to SQL은 편리하지만 위험할 수 있습니다.
- AI가 잘못된 SQL을 만들면 잘못된 데이터를 조회하거나, 과도한 조회를 하거나, 민감 정보를 노출할 수 있습니다.
- 따라서 다음 설계가 필요합니다.
 - 허용된 테이블만 조회하도록 제한
 - SELECT 쿼리만 허용
 - DELETE, UPDATE, INSERT, DROP 금지
 - 개인정보 컬럼 조회 제한
 - LIMIT 강제 적용실행 전 SQL 안전성 검사
 - 실행 결과를 그대로 보여주지 않고 요약·해석 단계 추가

Text to SQL 기본 흐름

사용자의 자연어 질문을 SQL로 변환하고, 안전성 검사를 거친 뒤 DB 결과를 해석해 답변하는 흐름입니다.



강의 설명 1

Text to SQL은 사용자가 SQL을 몰라도 자연어로 DB에 질문할 수 있게 하는 방식입니다.
예: "최근 7일 동안 불량률이 가장 높은 설비를 알려줘"

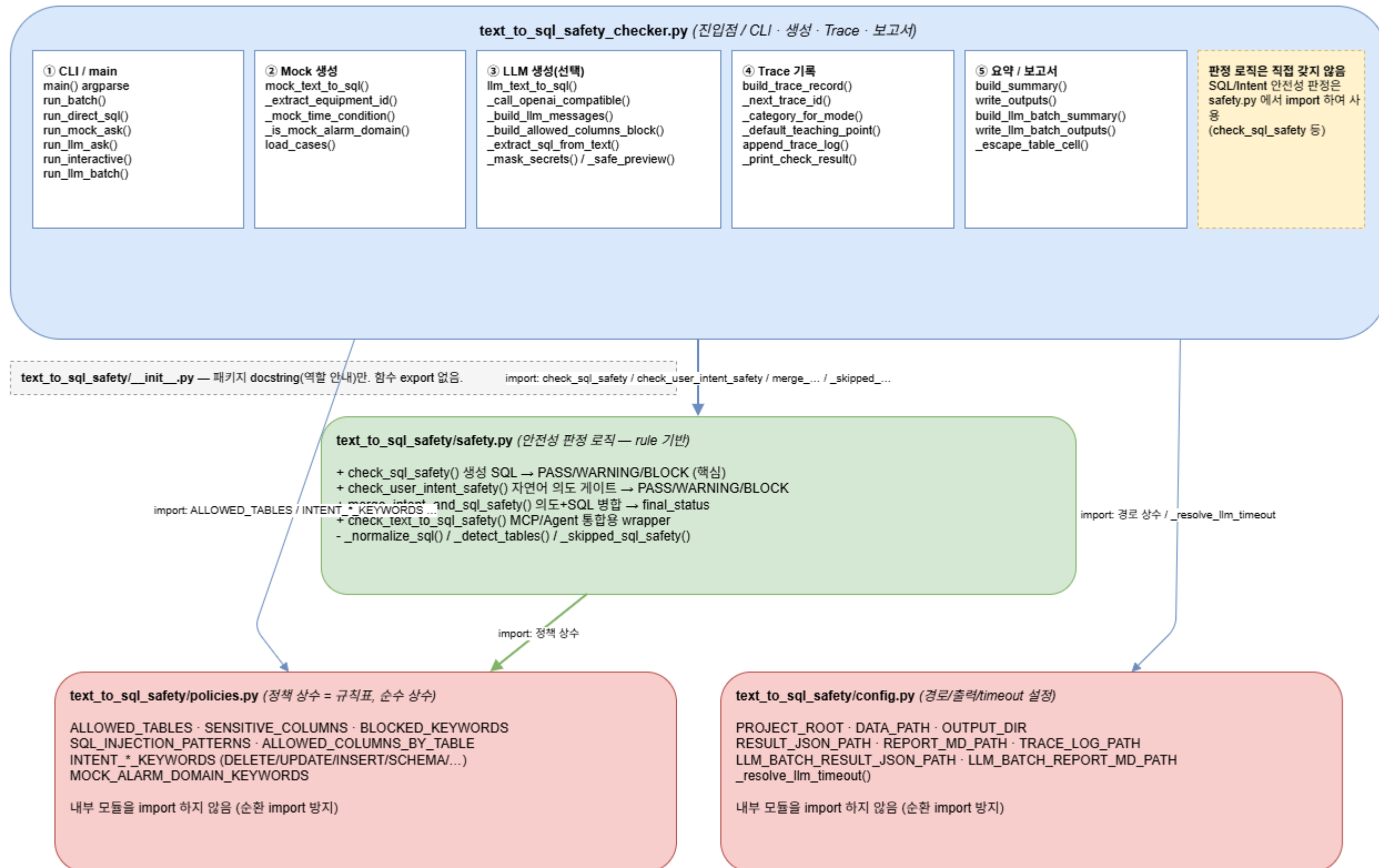
강의 설명 2

운영 환경에서는 SQL 생성보다 안전성 검사가 더 중요합니다.
허용 테이블, 금지 명령어, 개인정보 컬럼, LIMIT 적용 여부를 반드시 확인해야 합니다.

SQL 안전성 검사 예시

- SELECT 쿼리만 허용
- DELETE / UPDATE / INSERT / DROP 금지
- 개인정보 컬럼 조회 제한
- 과도한 전체 조회 방지를 위해 LIMIT 적용

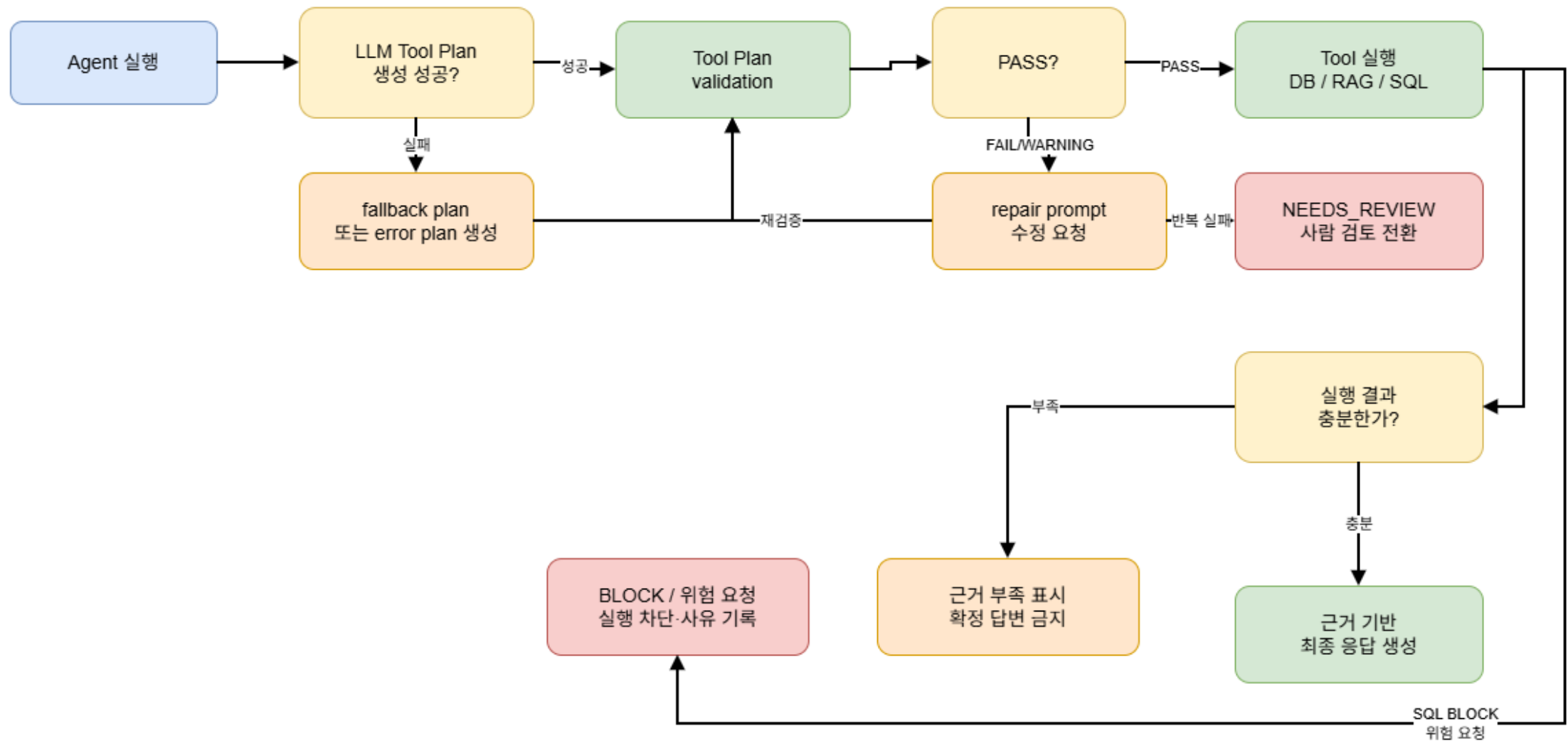
핵심 정리: Text to SQL은 자연어 질문을 SQL로 바꾸는 기술이며, 실제 서비스에서는 **SQL 안전성 검사**와 **결과 해석** 단계를 반드시 포함해야 합니다.



순서	핵심 함수	파일	역할
1	main()	text_to_sql_safety_checker.py	실행 옵션을 해석하고 Batch, 직접 SQL, mock 자연어, LLM 자연어, 대화형 모드 중 하나를 실행합니다.
2	load_cases()	text_to_sql_safety_checker.py	data/day4_text_to_sql_cases.json 테스트 케이스를 읽습니다.
3	mock_text_to_sql()	text_to_sql_safety_checker.py	자연어 질문을 규칙 기반 SQL로 변환합니다. LLM 없이 Text-to-SQL 흐름을 실행할 때 사용합니다.
4	llm_text_to_sql()	text_to_sql_safety_checker.py	자연어 질문을 LLM으로 SQL 변환합니다. LLM 실패 시 mock SQL로 fallback합니다.
5	check_user_intent_safety()	safety.py	SQL 생성 전에 사용자 질문 자체가 위험한 의도인지 검사합니다. 삭제, 수정, 개인정보 조회, 무제한 조회 등을 차단합니다.
6	check_sql_safety()	safety.py	생성된 SQL이 안전한지 검사합니다. SELECT 여부, 금지 키워드, SQL Injection, 민감 컬럼, LIMIT/WHERE 여부를 확인합니다.
7	merge_intent_and_sql_safety()	safety.py	사용자 의도 검사 결과와 SQL 검사 결과를 합쳐 최종 PASS / WARNING / BLOCK을 결정합니다.
8	run_batch()	text_to_sql_safety_checker.py	테스트 케이스 전체를 검사하고 결과 JSON/Markdown 보고서를 저장합니다.
9	run_llm_batch()	text_to_sql_safety_checker.py	테스트 케이스의 자연어 질문을 LLM으로 SQL 생성한 뒤, 의도 검사와 SQL 검사를 함께 수행합니다.

실패 상황별 대체 처리 흐름

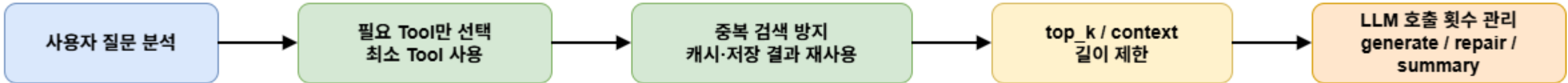
실패 상황별 대체 처리 흐름



원칙: 실패를 숨기지 않고, 빈 결과와 오류를 구분하며, 위험 요청은 실행하지 않고 사람 검토 또는 차단 흐름으로 전환합니다.

성능 최적화 및 비용 개선 점검

성능 최적화 및 비용 개선 점검



지연 원인 분리

- DB 지연
- RAG 검색 지연
- reranking 지연
- LLM 응답 지연
- repair 반복
- 과도한 context 구성

비용 증가 요인

- 불필요한 Tool 호출
- 같은 query 반복 검색
- 너무 큰 top_k
- 긴 문서 chunk 전체 전달
- 반복 repair
- 최종 요약 재호출 증가

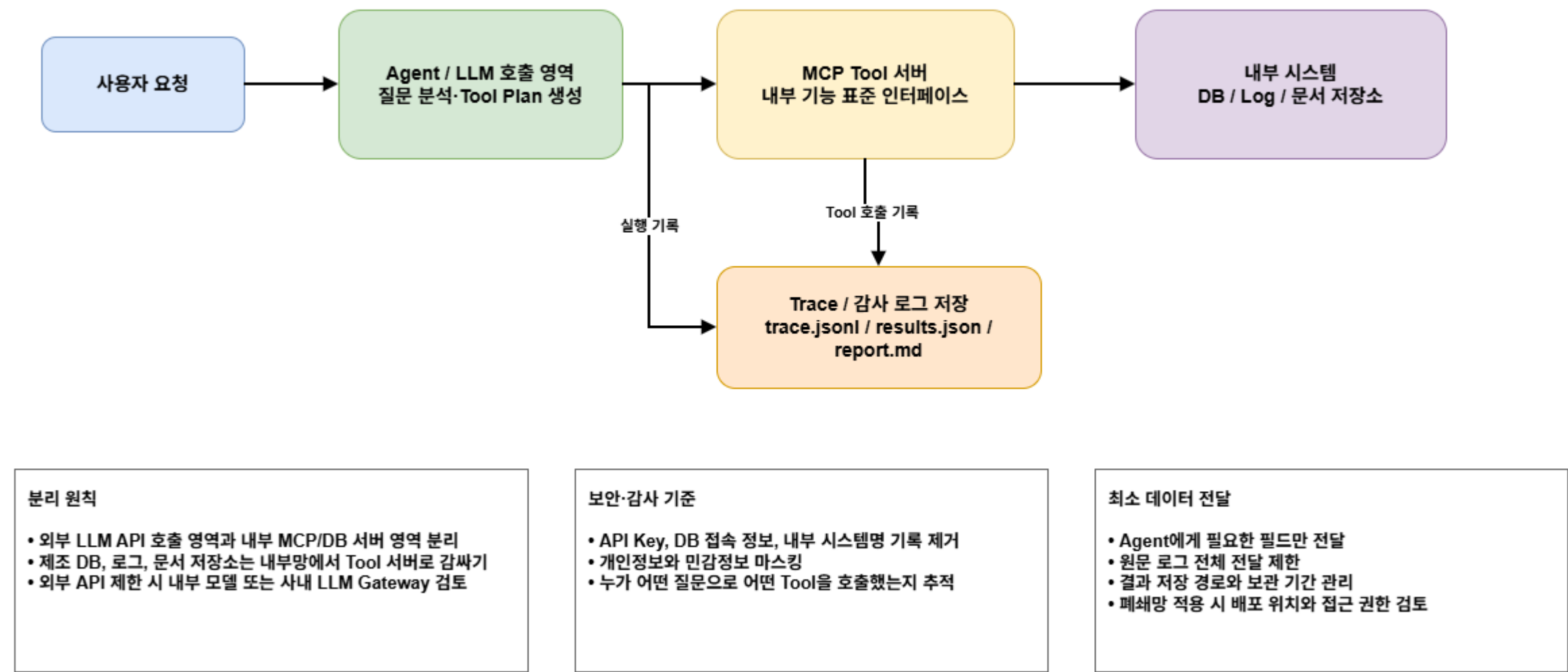
개선 방향

- 질문 유형별 Tool Call Plan 적용
- 단순 조회는 DB Tool 중심 처리
- 문서 근거 필요 시에만 search_manual 호출
- 검색 결과 개수 제한
- repair 반복 횟수 제한
- 자주 쓰는 결과 재사용 검토

정리: 성능 최적화는 속도만 높이는 작업이 아니라 필요한 근거만 효율적으로 수집하도록 Agent 흐름을 정리하는 작업입니다.

온프레미스 적용 시 평가·로그 설계 고려사항

온프레미스 적용 시 평가·로그 설계 고려사항



정리: 온프레미스 적용의 핵심은 내부 시스템을 안전하게 감싸고 실행 기록을 감사 가능한 구조로 남기는 것입니다.